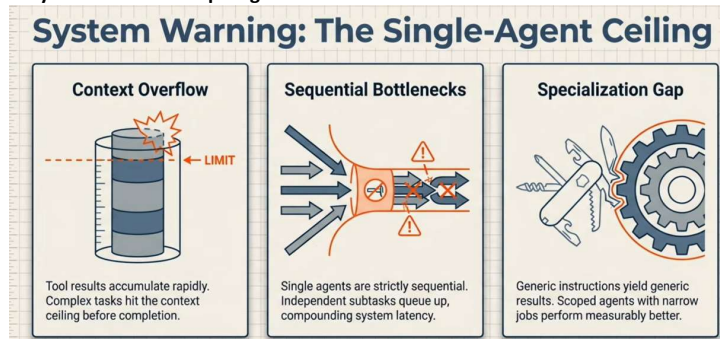


Chapter 2 - Sequential and parallel execution

05 September 2026 23:32

Why do we need multiple agents?



Context overflow -

Lot of tool calls happens so context overflows and context window is hit pretty soon. Even before task is finished, memory and chat history sent to agent/API may not be able to process it at the same time.

Sequential bottlenecks -

Single agent can use multiple tools at a time but it cannot use it parallelly. So if there are too many tools in the pipeline then sequential bottlenecks can also happen. Independent subtasks that could have run as a parallel agent itself now gets queued. And takes a lot of time to finish. So this compounds system latency.

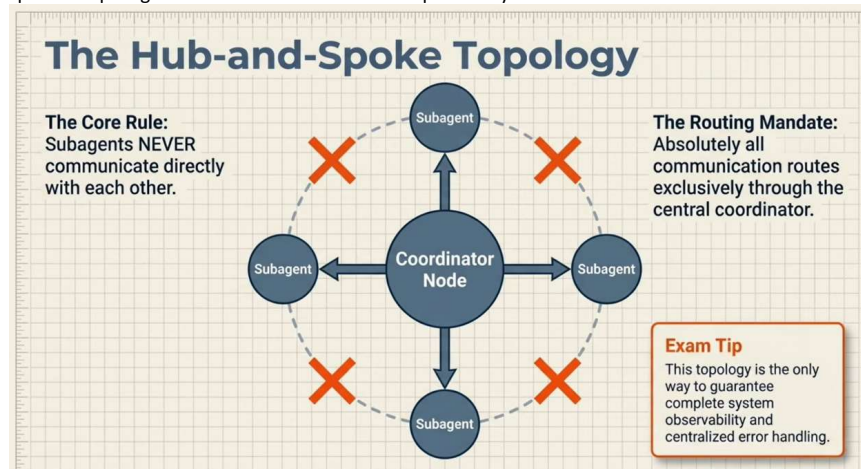
Specialization gap -

One agent cannot do everything. e.g., research agent cannot do work of analytics etc. hence it becomes like a generic agent.

Hence as per the solution, we deploy multi agent architecture specialized agents in parallel each with **scoped context**.

#The Hub and Spoke Topology

Subagents do not communicate with each other directly. They do not know each other. They are only dependent on the central agent. The single coordinator agent can span multiple agents and each one of them independently can finish their tasks.



Subagents are not responsible to generate final outputs. They will do their work and they report back to the coordinator. It takes all the things and then it demo it and present it.



Central agent decided - do I need an agent? When to invoke which sub agent? When should they respond etc.

Delegate task to appropriate subagent.

Result aggregation

It also manages error handling of the sub agents. Coordinator checks how can they handle the errored flag etc. handle them gracefully in an intelligent manner.

How does coordinator agent spans a new agent if at all it requires one?

It does through Task tool access.

Task Tool Permission Matrix

Configuration rules for allowedTools

Agent Role	Task Tool Access	Architectural Rationale
Coordinator	YES	Absolutely required. Without the Task tool, the coordinator cannot spawn any subagents.
Standard Subagent	NO	Standard workers must not spawn further agents. Uncontrolled recursive delegation breaks the system.
Hierarchical Sub-Coordinator	YES	Allowed only if this specific node is explicitly designed to delegate to a deeper, tertiary layer.

- If we need to spawn a sub-agent then we spawn it as a tool itself. The main agent spawns it like a tool. In Claude API that tool is more defined. The tool function returns some response.
- Whenever we are configuring a coordinator agent, we can add task to a list of allowed tools. It is a type of tool that we need to allow for a coordinator agent to use.
- It gives the ability to spawn the agent. Without it, it does not have the ability to spawn the sub agents. Without it, it cannot spawn anything.

Practical implementation in code format

When the coordinator agent calls a task it creates a new agent. Here tools type is given as "Task"

```

multi_agents.py > ...
1  # — Coordinator agent configuration —————
2  coordinator_config = {
3      "model": "claude-haiku-4-5",      # cost-efficient for orchestration
4      "system": """"You are a research coordinator.
5      Decompose tasks and delegate to specialized subagents.
6      Specify research goals and quality criteria.
7      Do NOT provide step-by-step procedures.""",
8
9      "tools": [
10         # — The Task tool enables subagent spawning —————
11         {
12             "type": "task",            # built-in Task tool type
13             "name": "Task",           # must be exactly "Task" (capital T)
14         },
15     ],
16     # — allowedTools MUST explicitly include "Task" —————
17     "allowedTools": ["Task", "compile_report"]
18 }
19
20 task_tool_call = {
21     "type": "tool_use",
22     "id": "tu_web_search_001",
23     "name": "Task",
24     "input": {
25         # — AgentDefinition fields —————
26         "description": "Web Search Specialist", # label
27         "prompt": """"Research goal: Find 5-8 recent papers (2022-2025)
28         on offshore wind energy environmental impact in the EU.
29         Quality criteria: peer-reviewed, include key findings,
30         flag conflicting conclusions. Return as structured JSON.""",
31
32         # — Subagent scoped to its role — NO Task tool here —————
33         "allowed_tools": ["web_search", "read_url"],
34
35         "model": "claude-haiku-4-5"
36     }
37 }

```

Then we should also specify the tool that was created into the "allowed_tools" section.
This was all the configuration of the coordinator_agent.

Let's talk about **task_tool_def**:

```

multi_agents.py > ...
20 task_tool_call = {
21     "type": "tool_use",
22     "id": "tu_web_search_001",
23     "name": "Task",
24     "input": {
25         # — AgentDefinition fields —————
26         "description": "Web Search Specialist", # label
27         "prompt": """"Research goal: Find 5-8 recent papers (2022-2025)
28         on offshore wind energy environmental impact in the EU.
29         Quality criteria: peer-reviewed, include key findings,
30         flag conflicting conclusions. Return as structured JSON.""",
31
32         # — Subagent scoped to its role — NO Task tool here —————
33         "allowed_tools": ["web_search", "read_url"],
34
35         "model": "claude-haiku-4-5"
36     }
37 }
38

```

Whenever coordinator agent has to spawn a new agent by itself, it calls the "task_tool_call".

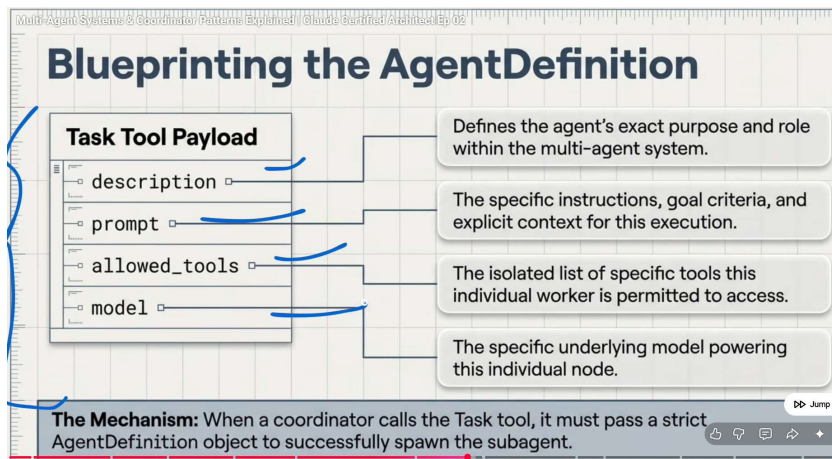
Inside task_tool_call, we provide input as 4 fields.

It is called the agent definition. Using this definition, coordinator spawns the agent.

- Description
- Prompt
- Allowed_tools
- Model

Task tool access hierarchy -

- Coordinator has the role for task_tool_Access.
 - Standard_subagent => These sub agents must not spawn further agents. Uncontrolled recursive delegation breaks the system.
 - Hierarchical sub-coordinator => Allowed only if this specific node is explicitly designed to delegate to a deeper, tertiary layer.
- (One Agent at the top, and there can be more coordinator called as sub-coordinator)

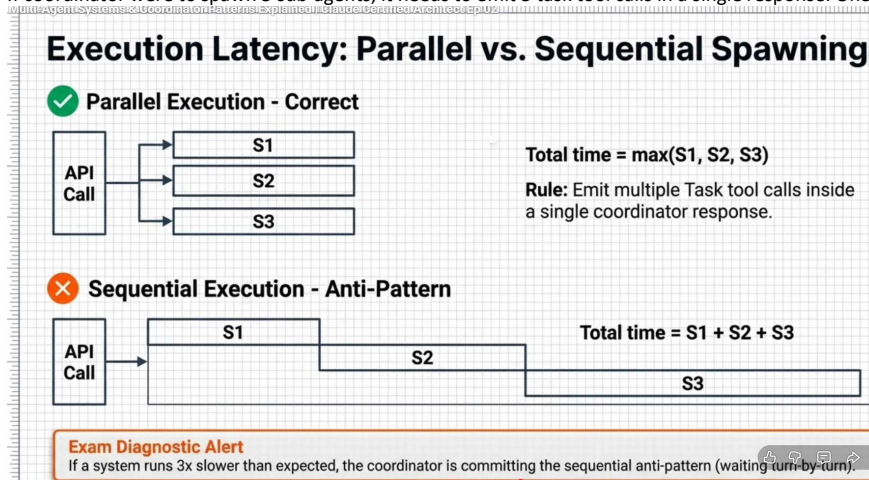


Execution Latency

Parallel vs Sequential Spawning

Coordinator can spawn multiple subagents at a same time in parallel. It need not be sequential.

If coordinator were to spawn 5 sub-agents, it needs to emit 5 task tool calls in a single response. One response, multiple tool calls.



If 3 agents are being spawned then the one that takes the most time to spawn, total time taken to spawn the agent will be $\max(s1, s2, s3)$

Sequential execution - It is not a good thing because if you want to do sequential agent then single agent can only do all the work. No point in spinning multiple agents unnecessarily.

#Exam prep:

If a system runs 3X slower than expected, the coordinator is committing sequential anti-pattern (waiting turn by turn)

In exam, they give scenarios, we need to check if a single coordinator response is it doing multiple tool call response or not? Check that in the payload.

Example:

4 sub agents

```

coordinator_response = {
  "role": "assistant",
  "content": [
    {"type": "text", "text": "Researching in parallel across 3 domains."},

    # — Task 1 —
    {"type": "tool_use", "id": "tu_env", "name": "Task",
     "input": {"description": "Environmental Researcher", ...}},

    # — Task 2 —
    {"type": "tool_use", "id": "tu_econ", "name": "Task",
     "input": {"description": "Economic Data Researcher", ...}},

    # — Task 3 —
    {"type": "tool_use", "id": "tu_policy", "name": "Task",
     "input": {"description": "EU Policy Analyst", ...}},
  ]
}

# + ALL THREE in the same content array = PARALLEL execution
# Separate turns = sequential anti-pattern

```

All this will have to follow an agent definition format -

```

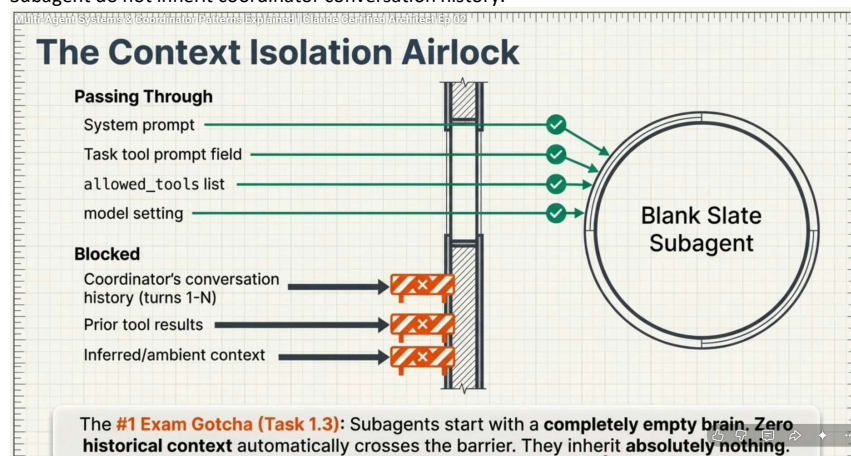
multi_agents.py > ...
25 # — AgentDefinition fields —
26 "description": "Web Search Specialist", # label
27 "prompt": "Research goal: Find 5-8 recent papers (2022-2025)
28 on offshore wind energy environmental impact in the EU.
29 Quality criteria: peer-reviewed, include key findings,
30 flag conflicting conclusions. Return as structured JSON.",
31
32 # — Subagent scoped to its role — NO Task tool here —
33 "allowed_tools": ["web_search", "read_url"],
34
35 "model": "claude-haiku-4-5"
36 }
37
38

```

The Context Isolation Airlock

Subagents always starts with a blank context.

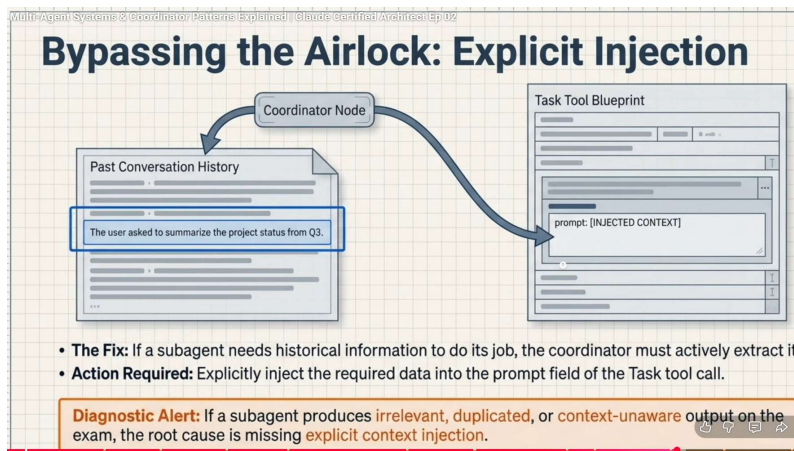
Subagent do not inherit coordinator conversation history.



But what if the subagent need some context ?

We need to bypass the airlock that has happened. It is the job of the coordinator to inject that information as per sub-agent need.

Coordinator does that in the task_tool call.



In the prompt field of the agent definition of the coordinator agent, we can inject that into the prompt field of the task tool call. Subagent also works with LLM so it all comes down to prompt.

#Exam prep: If the sub agent is not working as expected, maybe they will give you the agent definition . Solution: coordinator is not injecting the prompt field into the task_tool call.

Good prompting techniques -

Always mention the goal of the task, What you want to achieve etc.



Don't give step by step instruction.

Do not follow anti pattern. (basically do not do these things)

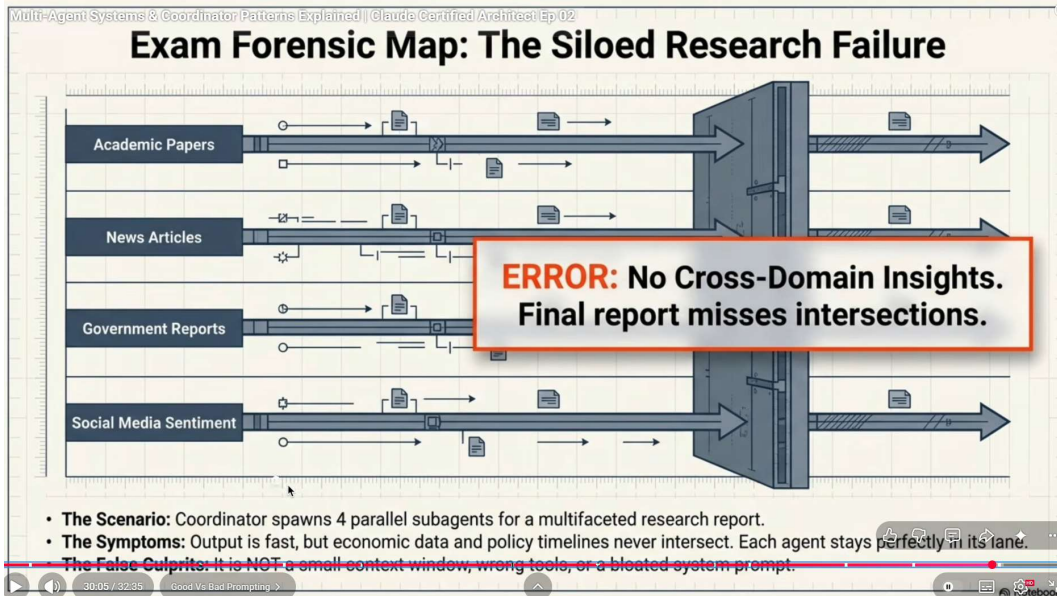
Anti pattern means negative prompting.

Rather follow best practices.

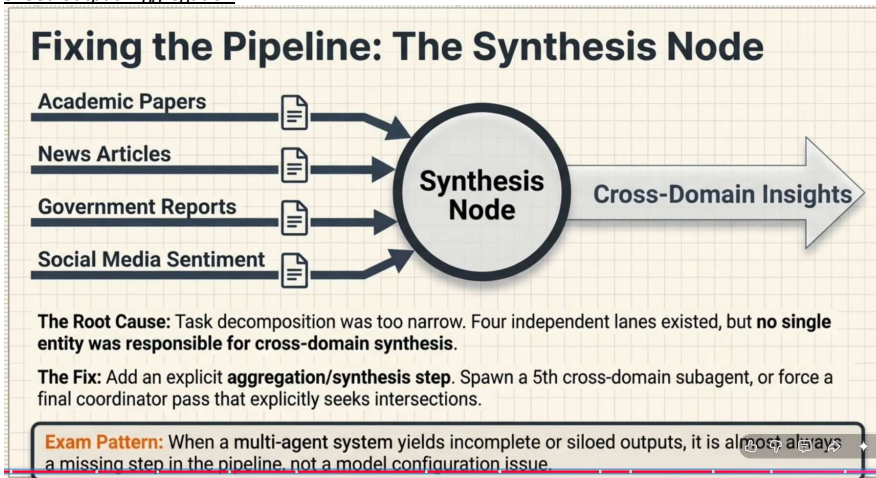
E.g., If we prompt saying only search this via Google scholar then If scholar is down then it means , LLM would either hallucinate or will not get you any results. Hence in those cases, we need to put some guardrails around.

So, we need to make sure that explicit context is getting injected rather than step by step injection.

Example scenario:



Siloed output: Aggregation



Here for example, all the 3 sub agents are enabled with context sharing. But the final aggregation or synthesis is not happening correctly. Cross domain insights was missing. Hence we need to explicitly pass the seek intersections. Coordinator must do this aggregation step. Either that or we can also create another sub agent that job itself is just to aggregate stuff. Depending on how we want the architecture, we can do these steps.

That is how multi agent system works.

Q to self: How do I understand in exam whether the issue is because of synthesis node issue or is it because of context sharing i.e., explicit Injection ?

Exam cheat sheet

Domain 1 Architecture Cheat Sheet

Concept	The Hard Rule	Exam Diagnostic (Why it matters)
Topology	All communication <u>routes</u> through the <u>coordinator</u> .	<u>Diagnostic: Enables absolute observability.</u>
Task Tool	Coordinator needs it; Subagents don't.	Diagnostic: Prevents recursive delegation loops.
Parallel Spawning	Multiple Task calls in ONE response.	Diagnostic: Fixes 3x latency issues.
Context Isolation	Subagents inherit zero history.	Diagnostic: Always explicitly inject context.
Siloed Outputs	Usually caused by overly narrow decomposition.	Diagnostic: Pipeline is missing a synthesis/aggregation step.